Announcements

1) HW 4 : due Oct 26

2) Quiz 2: Oct 29
   - topics:
     - nested loops / conditionals
     - arrays / 2D arrays
     - ArrayLists
     - mutable vs immutable objects
   - practice quiz with solutions will be posted by Fri Oct 23

Topics

1) Tic Tac Toe: how to approach a "bigger" programming problem.
2) Immutable Objects
3) Mutable Objects

Quick Review: 2D Arrays

```
int [][] nums = { {1, 2, 3},
                  {4, 5, 6},
                  {7, 8, 9},
                };
```

index for 5? [1] [1]

index for 1? [0] [0]

index for 7? [2] [0]

Tic Tac Toe

What do we need to represent a tic-tac-toe game?

- 3x3 board
- 2 players
- a way to make moves
- a way to determine whether someone has won

Break the Problem Down

1) create the board
2) display the board
3) get a move from a player
4) check that the move is valid
5) update the board
6) check for a winner
7) repeat until the game ends.

Break the Problem Down

1) `char [][] board = new char[3][3];`

2) `public static void displayBoard (char [][] board)`

3) `public static int[] getMoveFromPlayer (Scanner in)`

4) `public static boolean isValidMove(char [][] board, int row, int col)`

5) `public static void updateBoard (char [][] board, int row, int col, char player)`

6) `public static boolean checkWinner (char [][] board, char player)`

Let's Work Through a Small Piece!

- checking the win condition
- how do we check if a row has 3 X's?

```
char [][] board;

if (board[0][0] == 'X' &&
    board[0][1] == 'X' &&
    board[0][2] == 'X') {
    System.out.println("X wins!");
}
```

Generalize to Any Raw

```
static boolean checkRaw(
    char[][] board, int raw,
    char player) {
      if (board[raw][0] ==
          player &&
          board[raw][1] == player
          &&
          board[raw][2] == player) {
        return true;
      } else {
        return false;
      }
}
```

Check all 3 raws using a loop

```
static boolean checkRaws(
        char[][] board, char player) {
    for (int raw=0; raw<3; raw++) {
        if(checkRaw(board, raw, player) {
            // winning raw!
            return true;
        }
    }
    return false;
}
```

Remember: if a problem feels overwhelming, don't try to write the program all at once.

Instead, identify one small piece you understand how to solve, get it working, and build from there.

Primitives vs. References

```
int x = 5;
String name = "Milo";
int[] nums = {1, 2, 3};
```

int x = 5;

- primitive type
- var stores the value
- other primitive types: boolean, char, int, double...

Primitives vs. References

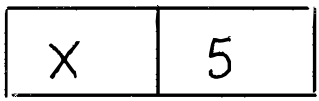
```
int x = 5;
String name = "Milo";
int[] nums = {1, 2, 3};
```

String name / int[] nums:

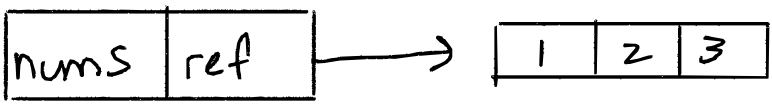
- reference types
- name stores a reference to a String object containing "Milo"
- nums stores a reference to an array object containing 1, 2, 3
- these vars refer to objects rather than storing the objects themselves

Primitives vs. References

```
int x = 5;
String name = "Milo";
int[] nums = {1, 2, 3};
```







Primitives vs. References

primitives: copying a value

```
int x = 5;
int y = x;

y = 10;

System.out.println(x);
   //prints 5
```

- y gets its own copy of the value
- changing y doesn't affect x

Primitives vs. References

references: copying a reference

```
int[] a = {1,2,3};
int[] b = a;
b[0] = 4;
System.out.println(a[0]);
   //prints 4
```

- a and b refer to the same array object
- changing the array through b changes a

Arrays are mutable: we can change their elements

Strings are immutable: we cannot change the contents of an existing String object

Class: defines a type of object

object: instance of a class

Attributes / fields: data associated with an object

methods: functions associated with an object

Ex: Point Class

```
import java.awt.Point;

class Point {
    int x;
    int y;
}
```

fields / attributes

instance of Point class

```
Point a = new Point(3, 4);

Point b = null;

print(a.x); // 3
print(b.x); // Null Pointer Exception
```

Ex: Point Class

```
Point a = new Point(3, 4);

Point b = null;

print(a.x); // 3
print(b.x); // Null Pointer Exception
```

* a references a Point object
* b does not reference any object
* null is a special keyword representing the absence of object reference
* primitive types like int can't be set to null

Null Keyword

```
Point a = new Point(3, 4);
Point b = null;
print(a.
```

Mutable Objects

```
import java.awt.Point;

Point p = new Point(3,4);

System.out.println(p.x);  // 3
System.out.println(p.y);  // 4

p.translate(2,3);

System.out.println(p);
    // (5,7)
```

Mutable Objects

1) what does new do?

2) what are x and y?

3) what does translate() do?

4) did we create a new object or update an existing one?

Mutable Objects: Aliasing

```
Point a = new Point(3,4);
Point b = a;
b.x = 10;
print(a.x); // 10
```

a and b refer to the same Point object

Reassignment vs. Mutation

```
Point a = new Point(3,4);

Point b = a;

b = new Point(10,20);

print(a.x); //3
print(b.x); //10
```

Why? `b = new Point(10,20);` creates a new Point object and assigns it to b.

Immutable Strings

```
String a = "Milo";

String b = a;

b = b.toUpperCase();

print(a); // Milo
print(b); // MILO
```

Why? toUpperCase() returns a new String rather than modifying the original.

Takeaway

Mutation: changes an object

Reassignment: changes which object a variable refers to.

Designing a Simple Class

Remember: an object is an instance of a class

```
public class Cat {
    String name;
    int age;

    public Cat(String name,
               int age) {
        this.name = name;
        this.age = age;
    }
    public void birthday() {
        age++;
    }
}
```

Designing a Simple Class

```
Cat milo = new Cat("Milo", 12);

milo.birthday();

print(milo.age); // 13
```

Important Take aways:

- fields
- constructors
- "this"
- instance methods

TODO: (ch 9)

- null Keyword
- command line args
- arg validation?
- wrapper classes?